

15.7

Resilience Patterns

Circuit Breaker, Retry, Bulkhead, Rate Limiter

Java Backend Architecture Interview Series • Tutorial Edition • Easy Diagrams & Examples

Why Resilience?

In microservices, every network call CAN and WILL fail. If Service A calls Service B and B is slow, A's threads block waiting. Soon A runs out of threads and becomes slow too. Then C, which calls A, fails. The whole system collapses — this is called a CASCADING FAILURE. Resilience patterns prevent this.

■ **Simple Analogy:** A circuit breaker in your home: when a wire overheats, the breaker trips and cuts power to prevent a fire. Once the problem is fixed, you flip it back on. Software circuit breakers do the same — cut calls to a failing service.

Pattern 1: Circuit Breaker

CLOSED (normal)	All calls go through. Counts failures. If failure rate > threshold (e.g., 50%), OPEN.
OPEN (fail fast)	Calls immediately return an error or fallback — no actual call made. Waits for timeout.
HALF-OPEN (probe)	After timeout, lets ONE call through. If it succeeds: CLOSE again. If it fails: stay OPEN.

```
// Resilience4j Circuit Breaker (Java)
@CircuitBreaker(name = "paymentService", fallbackMethod = "paymentFallback")
public PaymentResult processPayment(PaymentRequest req) {
    return paymentClient.charge(req); // this might fail
}

// Fallback runs when circuit is OPEN
public PaymentResult paymentFallback(PaymentRequest req, Exception ex) {
    return PaymentResult.queued("Payment queued, will retry shortly");
}
```

Pattern 2: Retry

Automatically retry a failed call. Use exponential backoff + jitter to avoid thundering herd.

```
@Retry(name = "inventoryService", fallbackMethod = "inventoryFallback")
public Stock getStock(String productId) {
    return inventoryClient.getStock(productId);
}

# Config: maxAttempts=3, waitDuration=500ms, multiplier=2 (500ms, 1s, 2s)
```

Pattern 3: Bulkhead

Isolate failures by giving each dependency its own thread pool. If the Payment pool is exhausted, it cannot steal threads from the Order pool.

Pattern 4: Rate Limiter

Limit how many requests a service can receive per second. Protects from overload and from API abuse.

Resilience4j Stack — All Patterns Together

Layer	Pattern	Config Example
1st (outer)	Rate Limiter	100 req/sec max
2nd	Circuit Breaker	OPEN at 50% fail rate
3rd	Retry	3 attempts, 500ms backoff
4th	Bulkhead	Thread pool size: 10
5th (inner)	Time Limiter	2s timeout, then fallback

Figure 7: Request passes through all layers; each protects from a different failure mode

■ **Real-World Example:** Netflix Hystrix (now Resilience4j): When the Recommendations service degrades, the Circuit Breaker opens and the fallback returns 'Top 10 movies' — not a blank screen. Bulkheads ensure a slow subtitle service never starves video delivery threads.